MGAIA Assignment 2 : BattleSnakes

March 2026

1 Introduction

In this assignment, we want you to familiarize yourself with Monte Carlo Tree Search (MCTS) and potentially explore different methodologies for implementing other agents. More specifically, the environment under investigation is Battlesnakes [1] with 4 agents, where the goal is to develop an agent that outlives the rest while also prioritizing its length.

2 Game overview

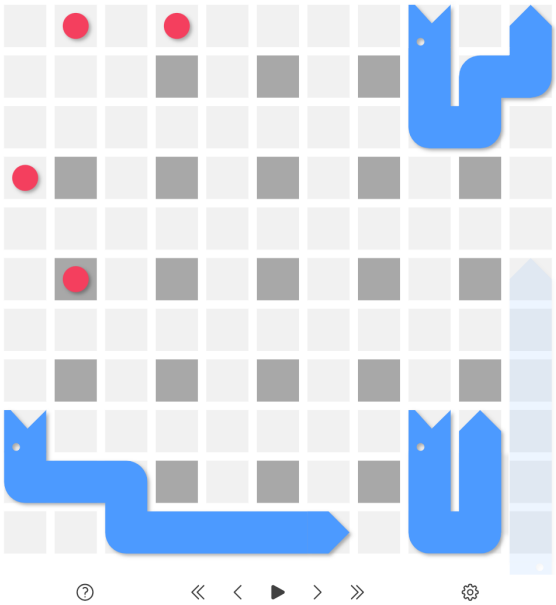

Figure 1: Three snakes with draining hazard pits which appear from turn 26

This is a fairly easy game that almost everyone is familiar with, the goal of this game is for the snake to stay alive as long as possible with an emphasis on snake length. Every turn the snake makes cost one health point, moving out of bounds costs 100, eliminating the snake. The starting length of each snake is 3 tiles, with a one tile increase for every food pellet consumed, while also reseting its health to 100. The length never decreases. There's always a minimum of 2 food pellets on the board at any given turn, with more constantly spawning. An agent has 1000ms to perform a turn, otherwise resulting in a timeout. They are also bound by a maximum turn limit of 300. When a snake collides with another head-on, the larger snake survives. If the snakes are of the same length, both are eliminated. Head-to-Body collisions result in the elimination of the colliding snake. Self collisions also result in elimination. Hazard pits appear from turn 26 and stack up every 25 turns, with 4 stacks in total. When fully stacked, it stays that way for 75 turns before fully draining. The first stack causes 14 points of damage to the snake, plus 1 point damage taken per turn. As stacks pile up, damage also piles piles by multiples of 14. A final score is calculated by giving 80% weightage to snake survival and 20% weightage to its length. The

[1] https://docs.battlesnake.com/

snake with the highest score wins. It is important to note that hazards only incur damage to the snake in a head on collision. Body of the snake passing through the hazards doesn't affect it.

3 Setup

- Have an up to date Python3 installed
- Install Flask 3.1.3
- Install the ***latest*** version of Go (disclaimer, using $apt install golang-go installs an out of date version that will not work)
- Install this GO package using
  $ *go install github.com/BattlesnakeOfficial/rules/cli/battlesnake@latest* (if this throws an error containing "cannot use path@version syntax in GOPATH mode" then your go version is too old)
- You can read more about the rules here https://docs.battlesnake.com/rules
- Download and extract the file (starter-snake-python.zip) from brightspace
- Go to that folder using a terminal and use the command $ *python main.py*, this launches the agent from Port 8000
- To test more snakes, open more terminals in the same folder and alter your port by using $ $*env* : $PORT = 8001$ (for windows) $ *export* $PORT = 8001$ (for linux) and then run $*python main.py* and it should state the snake is running on port 8001 now.
- You can always adjust the number of snakes you want to test by modifying "run_game.py", the default is 4 snakes. Adjust the snakes and their respective ports by altering the CMD portion at the top of the "run_game.py", for example, to use just a single snake comment out the lines
  "–name", "Snake2", "–url", "http://127.0.0.1:8001",
  "–name", "Snake3", "–url", "http://127.0.0.1:8002",
  "–name", "Snake4", "–url", "http://127.0.0.1:8003",
- To run the game, in a separate terminal, use the command *python run_game.py*. Make sure that all ports in the "run_game.py" file are at that time running a snake program, otherwise it will throw an error.

4 Task

Your task is to familiarize yourself with BattleSnakes, analyze the game, design and implement a solution that uses MCTS, and create a scientific report that describes your solution-finding process. In detail:

- Familiarize yourself with the BattleSnakes Game
  - Don't forget to mention your analysis, assumptions and simplifications about the environment in the report!
- Implement a heuristic agent / write a state evaluation function
- Implement an MCTS agent
  - actions need to be executed by your MCTS agent! Do not hardcode individual actions (e.g. at the start).
  - What assumptions and changes do you have to apply to vanilla MCTS to get it working with the provided environment?
  - Experiment and report on the effects of MCTS hyperparameters
  - Use your heuristic state evaluation to guide your simulation steps. What do you notice compared to random rollouts?

  - Look at and implement at least two improvements for the other MCTS stages. For example, you could exchange UCB for Rapid Value Action Estimation[3]. The following publication can give you more inspiration on other variations to try[4].
  - In your report, include a section called Future Work where you highlight other possible improvements not included in your implementation. Be sure to base these on related scientific **literature!**
  - Neural Networks / Reinforcement Learning are forbidden as variation in any MCTS stage! This assignment is supposed to teach you MCTS and its caveats, not neural network training on games! If you really want to, you can develop a project in A3 that will give you this opportunity

- Compare the performance of your heuristic agent(s) and MCTS variations (e.g. random rollouts, heuristic rollouts, etc.) by running a local tournament and portray its results. What can you observe?
  - Be sure to run enough matches to have statistically relevant results!
  - Do not only look at win rate %. If you compare multiple agents in a tournament you should use a scoring mechanism such as ELO or TrueSkill[1]!
  - Manually analyze matches to review your agents skills. What kind of strengths and weaknesses emerge from the different adjustments you apply to your agents?

5 Submission

5.1 Report

Make sure to nicely document everything that you do in the report. Your final submission consists of:

- Python source code for each of your agents, i.e. one file per agent.
- A self-contained scientific pdf report with figures, references, etc. The page count we expect of you might vary depending on your layout. This report contains an explanation of the techniques you applied

If you have any questions about this assignment, please visit our lab sessions on Thursdays where we can help you out. In case you cannot make it in person, you can post questions about the contents of the course on the Discord discussion forum, where other students can also read and reply to your questions.

The deadline for this assignment is the 03.04.2026 at 23:59.

5.2 Contest

We will also be holding a tournament in one of the labs where one of your agents will compete against other teams. You can only submit one of your agents, so choose wisely. Submit your agent as teamname.py in the separate Brightspace submission "Contest Agent". This agent version may not have a thought time exceeding one second! In case there are too many groups / execution takes too long we may finish gathering the results outside of the lab.

6 Grading Criteria

- 50 % Quality of your solution concept (correct MCTS implementation, heuristic / state evaluation, simulation steering adjustment, other MCTS improvements, parameter tuning, agent skill analysis)
- 20% Tournament (see 4) (value compared by, statistical relevance / runs, amount of agent variations included, result presentation)
- 20% Report
- 10% Score based on Course Tournament placement (see 5.2)

7 References

References

[1] Ralf Herbrich, Tom Minka, and Thore Graepel. "TrueSkill™: a Bayesian skill rating system". In: *Proceedings of the 20th International Conference on Neural Information Processing Systems*. NIPS'06. Canada: MIT Press, 2006, pp. 569–576.

[2] Aja Huang et al. "Mastering the game of Go with deep neural networks and tree search". In: *Nature* 529 (Jan. 2016), pp. 484–489. DOI: 10.1038/nature16961.

[3] Arpad Rimmel, Fabien Teytaud, and Olivier Teytaud. "Biasing Monte-Carlo Simulations through RAVE Values". In: *Computers and Games*. Ed. by H. Jaap van den Herik, Hiroyuki Iida, and Aske Plaat. Berlin, Heidelberg: Springer Berlin Heidelberg, 2011, pp. 59–68. ISBN: 978-3-642-17928-0.

[4] Maciej Świechowski et al. "Monte Carlo Tree Search: a review of recent modifications and applications". In: *Artificial Intelligence Review* 56.3 (July 2022), pp. 2497–2562. ISSN: 1573-7462. DOI: 10.1007/s10462-022-10228-y. URL: http://dx.doi.org/10.1007/s10462-022-10228-y.